

Day 1: Save Password

1. Day Number

Day 1 of the Python foundation plan

2. Topic Name

Variables, comments, indentation, input, and output

Today you will learn how to:

- Receive text from a user with `input()`
 - Store that text in a variable
 - Display a message with `print()`
 - Write comments that explain code
 - Recognize why indentation matters in Python
-

3. Connection with Previous Learning

This is the first step of the learning plan, so there is no previous program to extend.

You are beginning a simple login system. Before a password can be checked, changed, or used for login, the program must first receive and store it.

The password will only remain in memory while the program is running. Permanent storage will be introduced later through file handling.

4. Important Topics

Variable

A variable is a named place used to store a value.

```
message = "Welcome"
```

Here:

- `message` is the variable name.
- `"Welcome"` is the stored value.
- `=` assigns the value to the variable.

Comment

A comment is a note for people reading the code. Python ignores it.

```
# Ask the user for their name
```

Indentation

Indentation means spaces at the beginning of a line. Python uses indentation to show which statements belong inside a block such as a function.

```
def greet():  
    print("Hello")
```

Input

`input()` pauses the program and waits for the user to type something.

```
name = input("Enter your name: ")
```

The value returned by `input()` is text.

Output

`print()` displays information on the screen.

```
print("Details saved successfully.")
```

5. Foundational Notes

Variables remember values

When a value is assigned to a variable, it can be used later in the program.

```
city = "Pune"  
print(city)
```

Use meaningful variable names

Names such as `password`, `user_name`, and `account_balance` explain what the values represent.

Avoid unclear names such as:

```
x = "secret"
```

`input()` produces a string

Even when a user enters numbers, `input()` initially returns a string. A password should remain a string because it may contain letters, numbers, spaces, or symbols.

`=` does not mean comparison

On Day 1, `=` is used to assign a value:

```
color = "blue"
```

Comparison with `==` will be introduced on Day 2.

Do not print the password

The task only requires a success message. Displaying the entered password would expose private information.

Empty input and spaces

For this first exercise, the program only stores the value. It does not yet reject an empty password or remove spaces. Validation will be introduced gradually in later lessons.

6. Easy Example

Ask a user for a favorite color and store it:

```
favorite_color = input("Enter your favorite color: ")
print("Favorite color saved.")
```

If the user enters:

```
Blue
```

The variable `favorite_color` stores "Blue", and the program displays:

```
Favorite color saved.
```

This example demonstrates the same input-store-output pattern needed for today's password exercise, but it is not the complete solution.

7. Problem Statement

Create a beginner-friendly Python program that:

1. Asks the user to create a password.
2. Saves the entered password in a variable.
3. Prints a success message after the value has been stored.
4. Does not display the password itself.

Use a functional approach to organize the program.

8. Concepts Used

- Comment
 - Function
 - Indentation
 - Variable
 - Assignment operator =
 - String
 - `input()`
 - `print()`
-

9. Thought Process

Step 1: Identify the required input

The program needs one value from the user: a password.

Step 2: Choose a meaningful variable name

Use a name that clearly represents the entered value, such as `password`.

Step 3: Store the input

Use `input()` and assignment so the password is saved in the variable.

Step 4: Confirm that the action completed

Use `print()` to show a success message.

Step 5: Protect the entered value

Do not include the password variable in the output.

Step 6: Organize the steps

Place the main actions inside a small function and call the function so the program runs.

10. Beginner-Friendly Pseudocode

```
START

DEFINE a function for saving a password
    ASK the user to create a password
    STORE the entered text in a password variable
    DISPLAY a password-saved success message

CALL the function

END
```

Pseudocode describes the logic in plain language. It does not need exact Python syntax.

11. Suggested Solving Approach: Functional Approach

Create one small function responsible for:

1. Receiving the password
2. Storing it in a local variable
3. Printing the success message

After defining the function, call it once.

The function structure will look like this:

```
def function_name():
    # Indented instructions go here

# Call the function here
```

Complete the instructions yourself using `input()`, a variable, and `print()`.

12. Easy Edge Cases

Empty password

The user may press Enter without typing anything. The variable will contain an empty string:

```
" "
```

For Day 1, simply observe this behavior. Do not add advanced validation yet.

Password containing spaces

A password may contain spaces:

```
my secret password
```

`input()` stores those spaces as part of the string. Do not remove them automatically in this exercise.

13. Expected Input

One password entered by the user.

Example:

```
Create your password: hello123
```

14. Expected Output

After receiving the password, display a confirmation such as:

```
Password saved successfully.
```

The actual password must not appear in the output.

15. Hint Only

Use this pattern:

```
variable_name = input("Your message: ")  
print("Success message")
```

Adapt the variable name and messages for a password, place the statements inside a function, and remember to call the function.

Do not copy a complete solution. Try writing it yourself from the pseudocode.